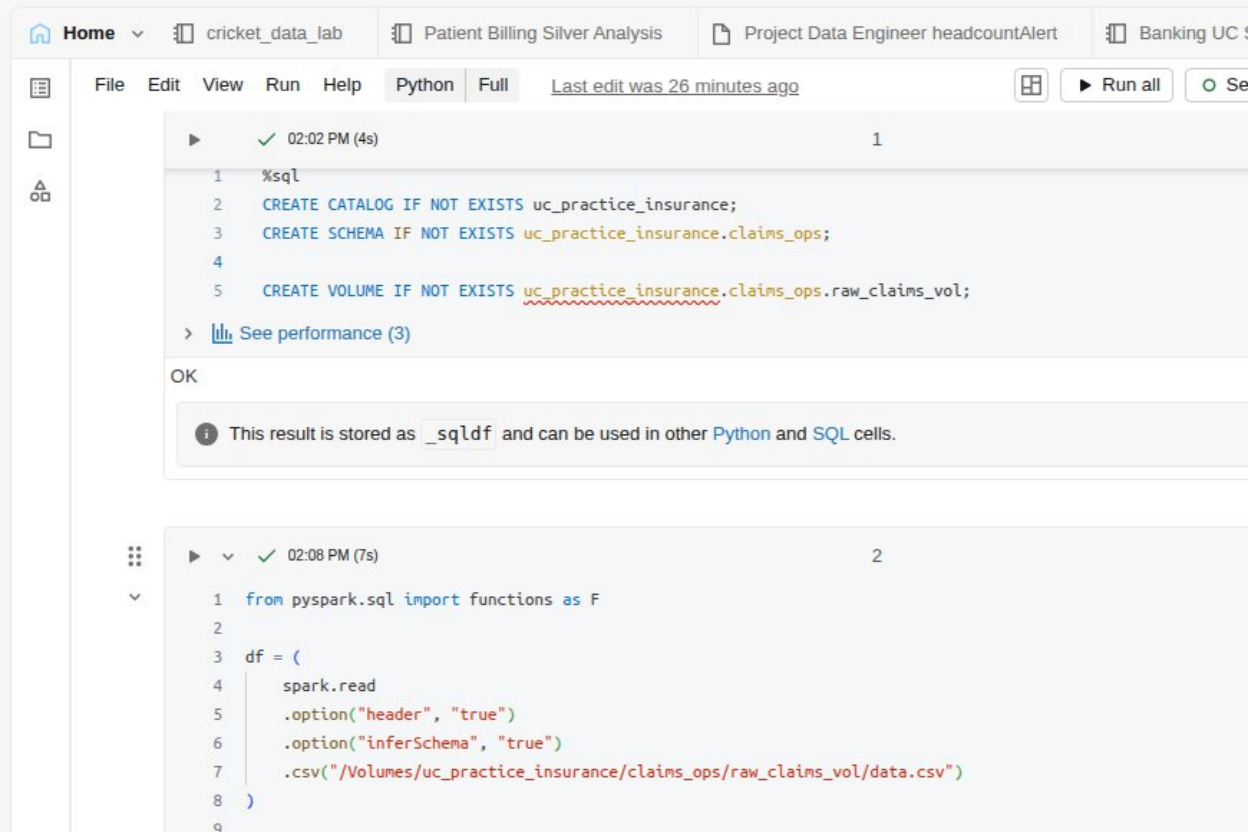
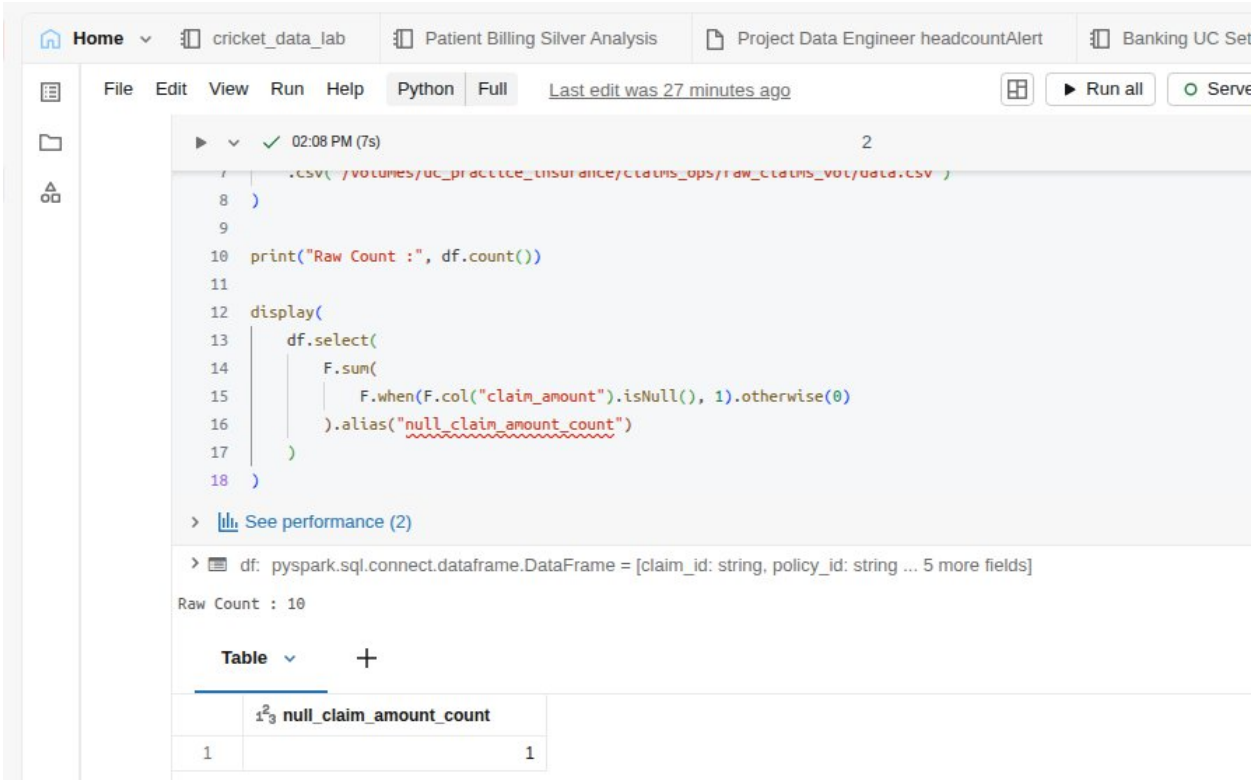
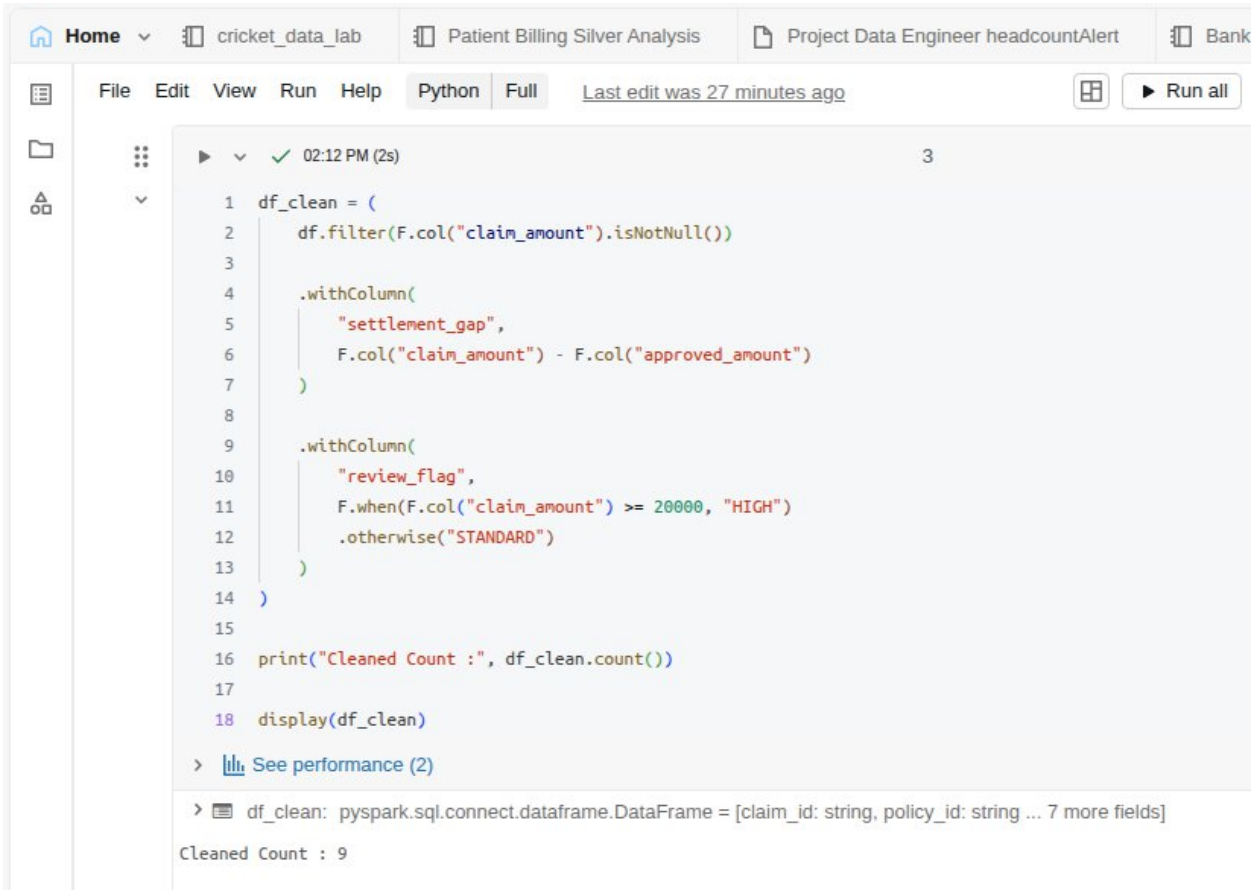
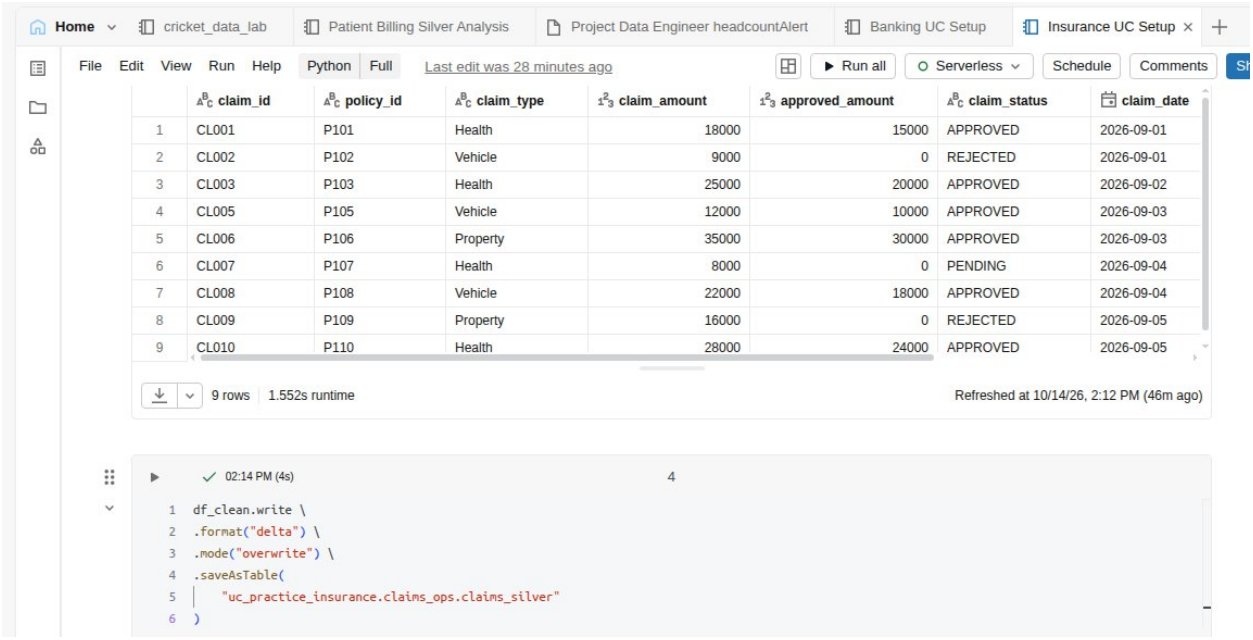
WEEK 9 DAY 3 COD ASSIGNMENT TASKS OUTPUTS :

OUESTION : INSURANCE(MEDIUM)

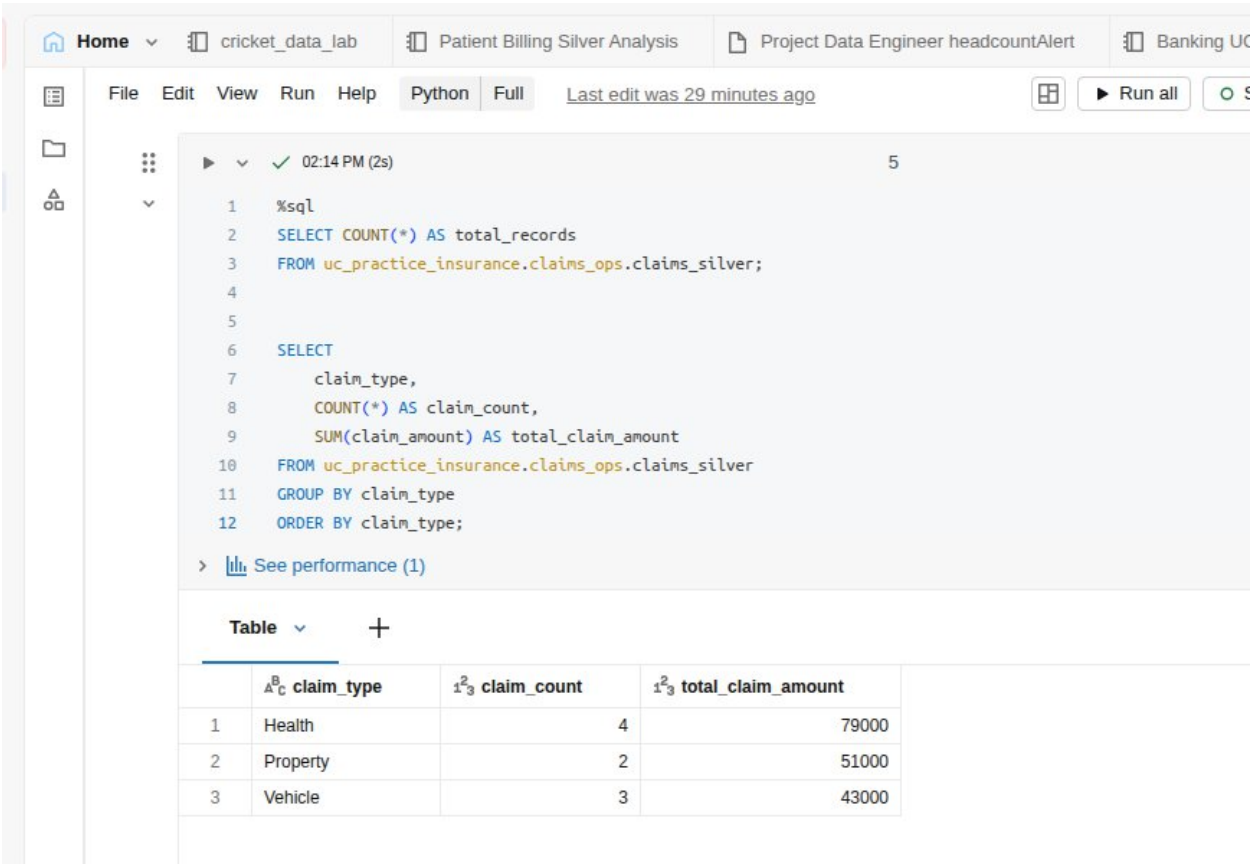



Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking UC Set
File Edit View Run Help Python Full
Last edit was 27 minutes ago
Run all
Serve
02:08 PM (7s)
2
.csv("/Volumes/uc_practice_insurance/claims_ops/raw_claims_vol/data.csv")
)
print("Raw Count :", df.count())
display(
df.select(
F.sum(
F.when(F.col("claim_amount").isNull(), 1).otherwise(0)
).alias("null_claim_amount_count")
)
)
See performance (2)
df: pyspark.sql.connect.dataframe.DataFrame = [claim_id: string, policy_id: string ... 5 more fields]
Raw Count : 10
Table
null_claim_amount_count
1
1

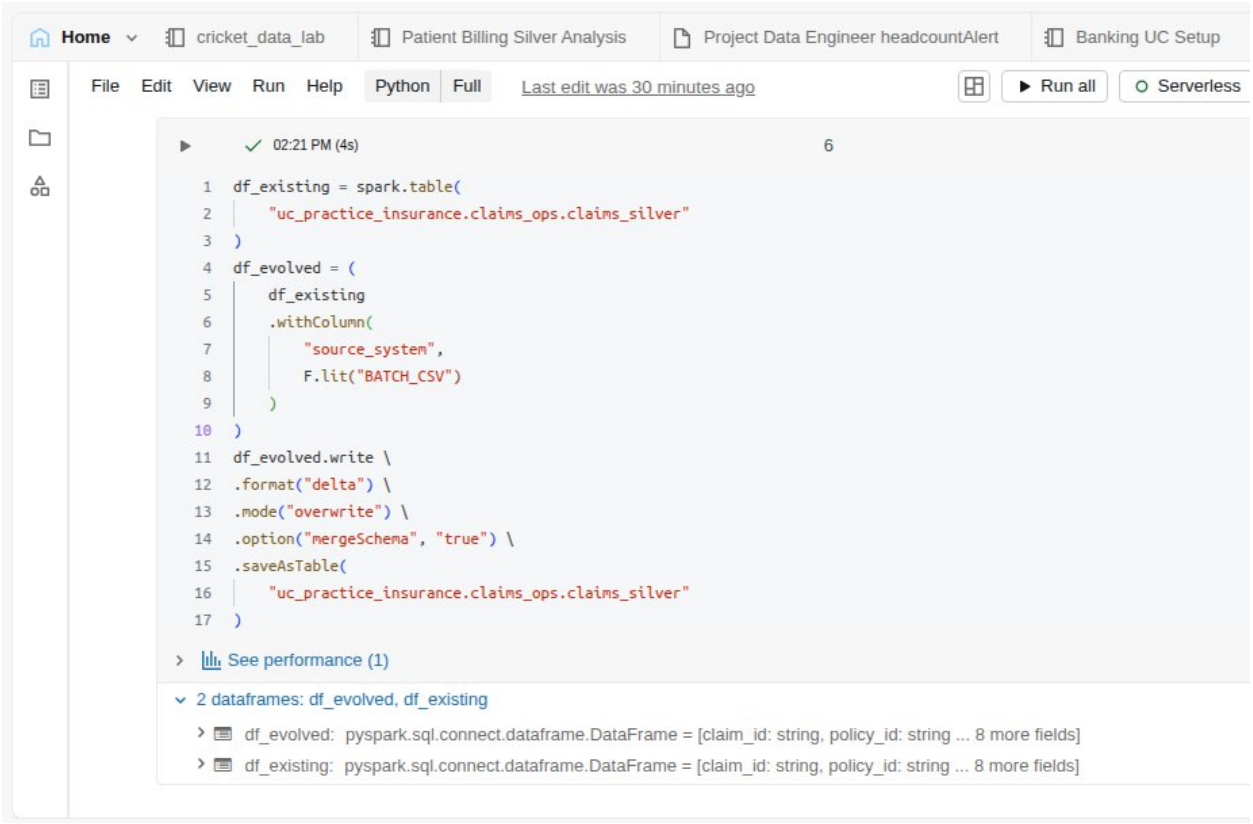

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Bank
File Edit View Run Help Python Full
Last edit was 27 minutes ago
Run all
02:12 PM (2s)
3
df_clean = (
df.filter(F.col("claim_amount").isNotNull())
.withColumn(
"settlement_gap",
F.col("claim_amount") - F.col("approved_amount")
)
.withColumn(
"review_flag",
F.when(F.col("claim_amount") >= 20000, "HIGH")
.otherwise("STANDARD")
)
)
print("Cleaned Count :", df_clean.count())
display(df_clean)
See performance (2)
df_clean: pyspark.sql.connect.dataframe.DataFrame = [claim_id: string, policy_id: string ... 7 more fields]
Cleaned Count : 9

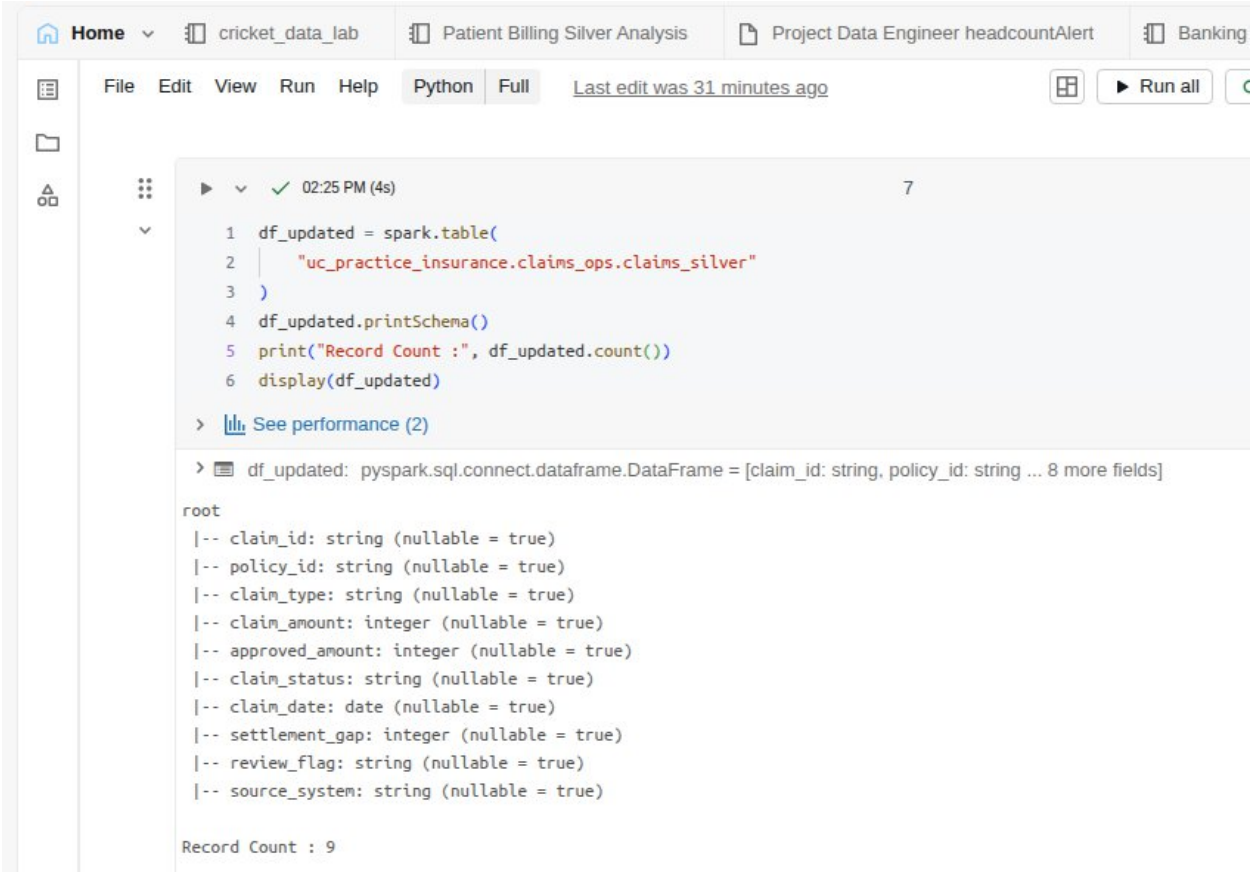


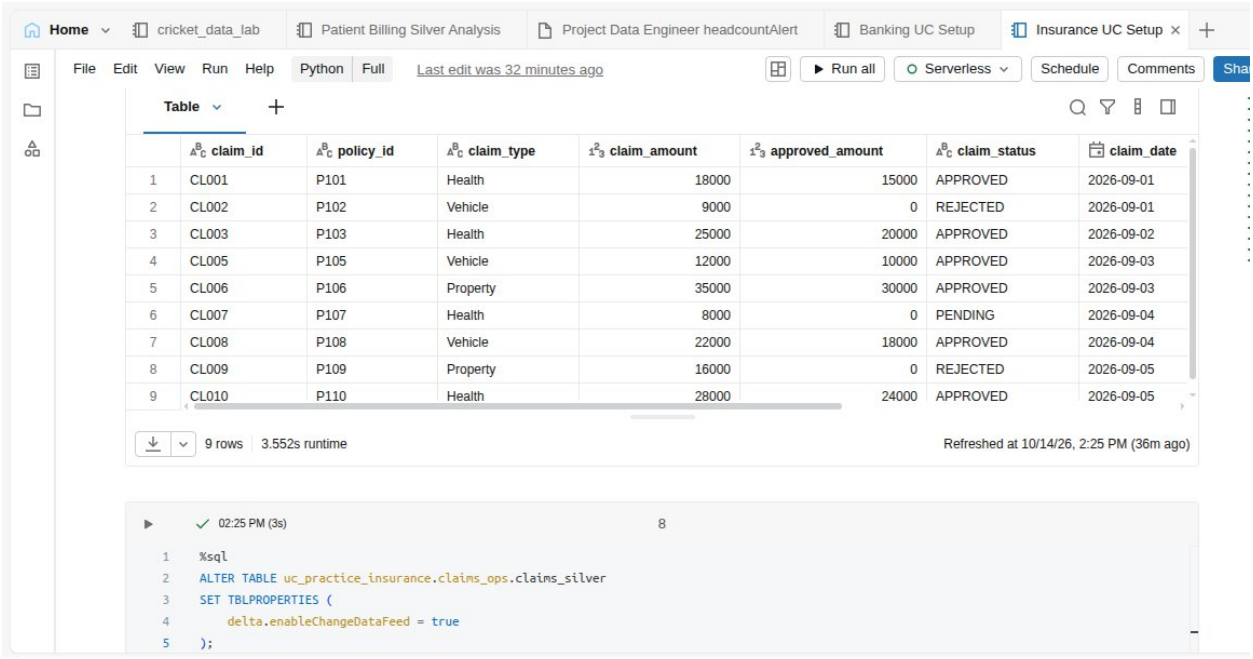
	claim_id	policy_id	claim_type	claim_amount	approved_amount	claim_status	claim_date
1	CL001	P101	Health	18000	15000	APPROVED	2026-09-01
2	CL002	P102	Vehicle	9000	0	REJECTED	2026-09-01
3	CL003	P103	Health	25000	20000	APPROVED	2026-09-02
4	CL005	P105	Vehicle	12000	10000	APPROVED	2026-09-03
5	CL006	P106	Property	35000	30000	APPROVED	2026-09-03
6	CL007	P107	Health	8000	0	PENDING	2026-09-04
7	CL008	P108	Vehicle	22000	18000	APPROVED	2026-09-04
8	CL009	P109	Property	16000	0	REJECTED	2026-09-05
9	CL010	P110	Health	28000	24000	APPROVED	2026-09-05

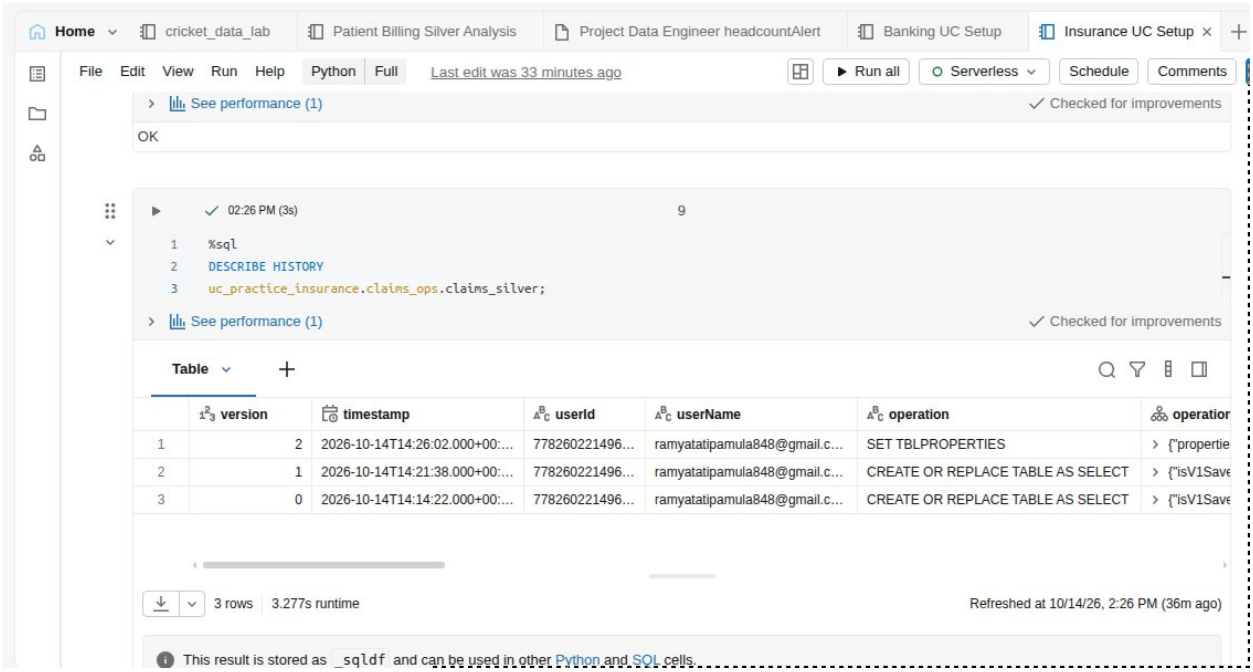


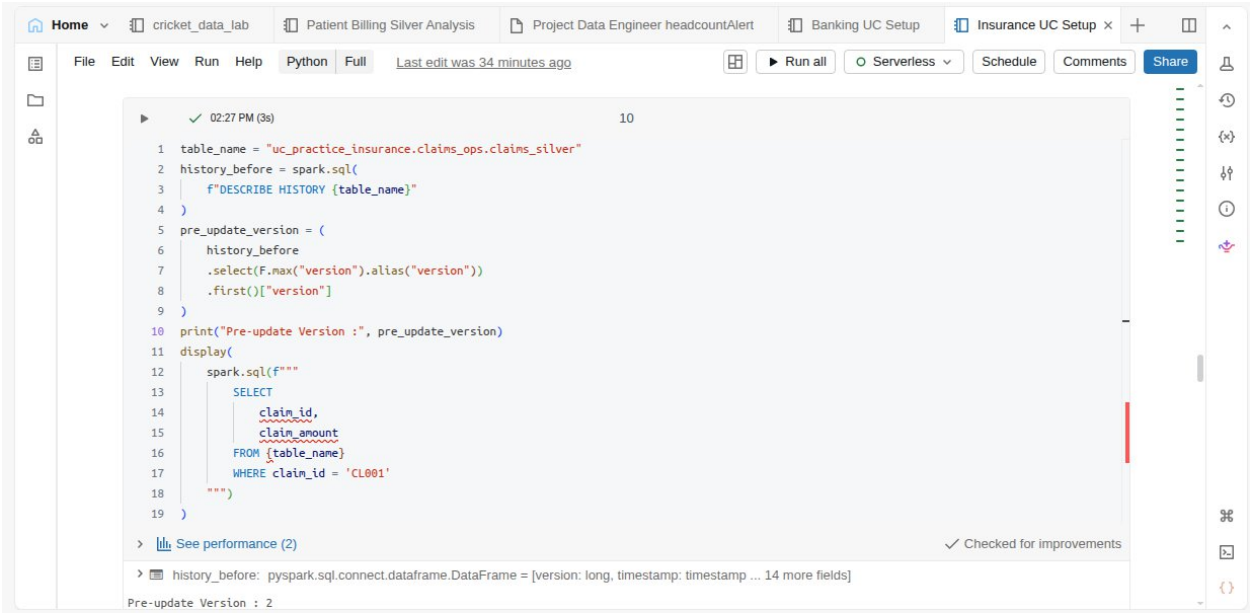
	claim_type	claim_count	total_claim_amount
1	Health	4	79000
2	Property	2	51000
3	Vehicle	3	43000

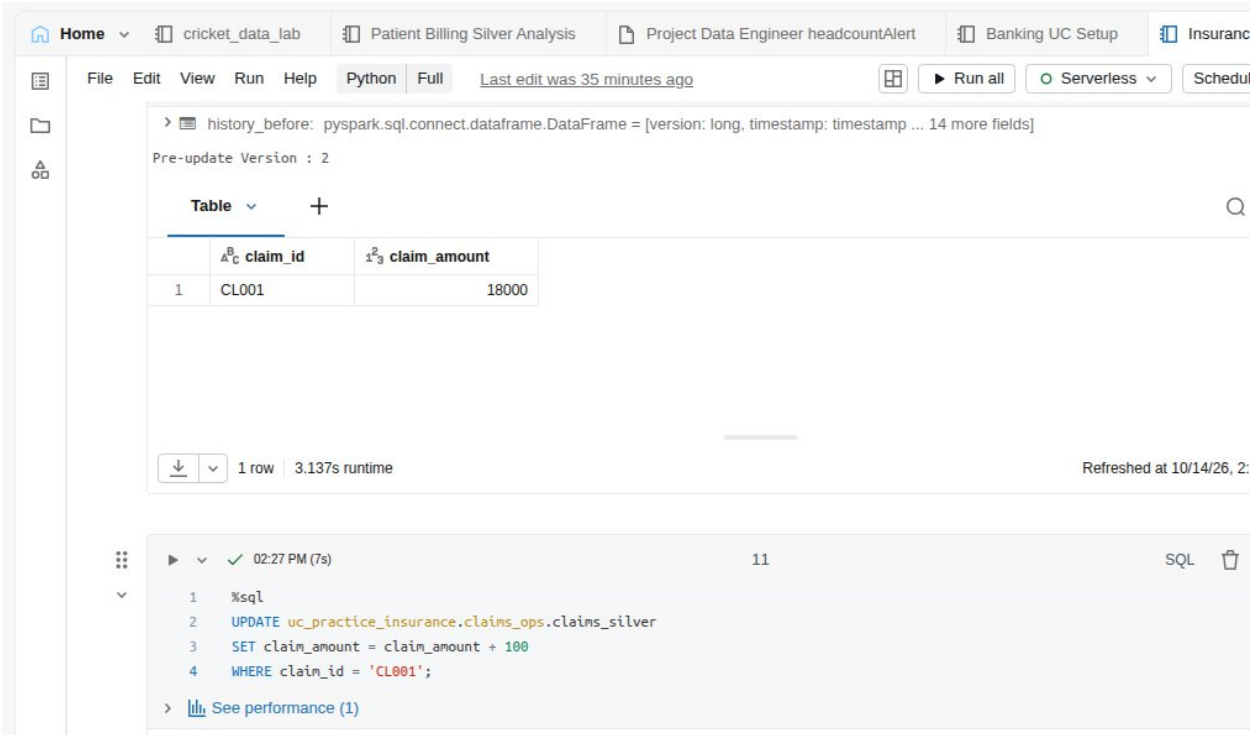

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking UC Setup
File Edit View Run Help
Python Full
Last edit was 30 minutes ago
Run all
Serverless
02:21 PM (4s)
6
df_existing = spark.table(
"uc_practice_insurance.claims_ops.claims_silver"
)
df_evolved = (
df_existing
.withColumn(
"source_system",
F.lit("BATCH_CSV")
)
)
df_evolved.write \
.format("delta") \
.mode("overwrite") \
.option("mergeSchema", "true") \
.saveAsTable(
"uc_practice_insurance.claims_ops.claims_silver"
)
See performance (1)
2 dataframes: df_evolved, df_existing
df_evolved: pyspark.sql.connect.dataframe.DataFrame = [claim_id: string, policy_id: string ... 8 more fields]
df_existing: pyspark.sql.connect.dataframe.DataFrame = [claim_id: string, policy_id: string ... 8 more fields]

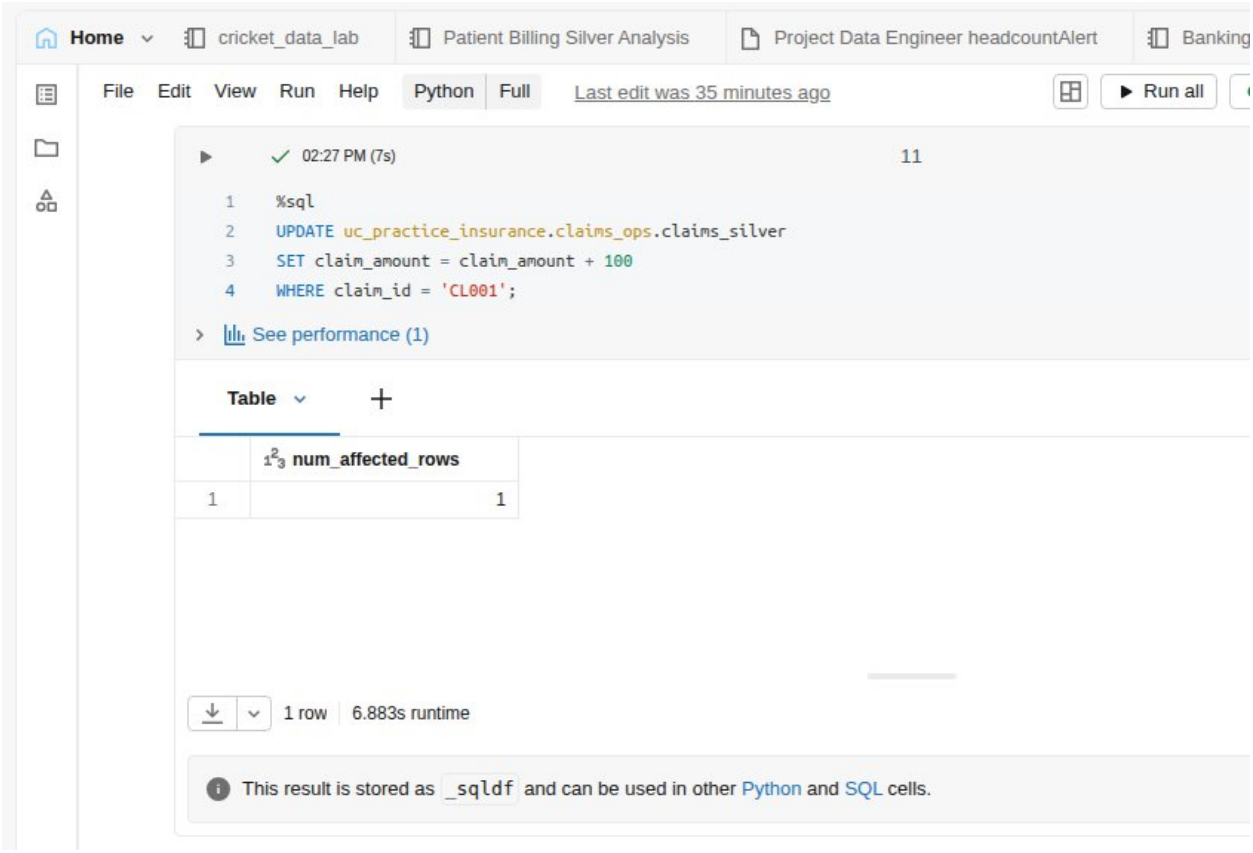

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking
File Edit View Run Help
Python Full
Last edit was 31 minutes ago
Run all
02:25 PM (4s)
7
df_updated = spark.table(
"uc_practice_insurance.claims_ops.claims_silver"
)
df_updated.printSchema()
print("Record Count :", df_updated.count())
display(df_updated)
See performance (2)
df_updated: pyspark.sql.connect.dataframe.DataFrame = [claim_id: string, policy_id: string ... 8 more fields]
root
|-- claim_id: string (nullable = true)
|-- policy_id: string (nullable = true)
|-- claim_type: string (nullable = true)
|-- claim_amount: integer (nullable = true)
|-- approved_amount: integer (nullable = true)
|-- claim_status: string (nullable = true)
|-- claim_date: date (nullable = true)
|-- settlement_gap: integer (nullable = true)
|-- review_flag: string (nullable = true)
|-- source_system: string (nullable = true)
Record Count : 9

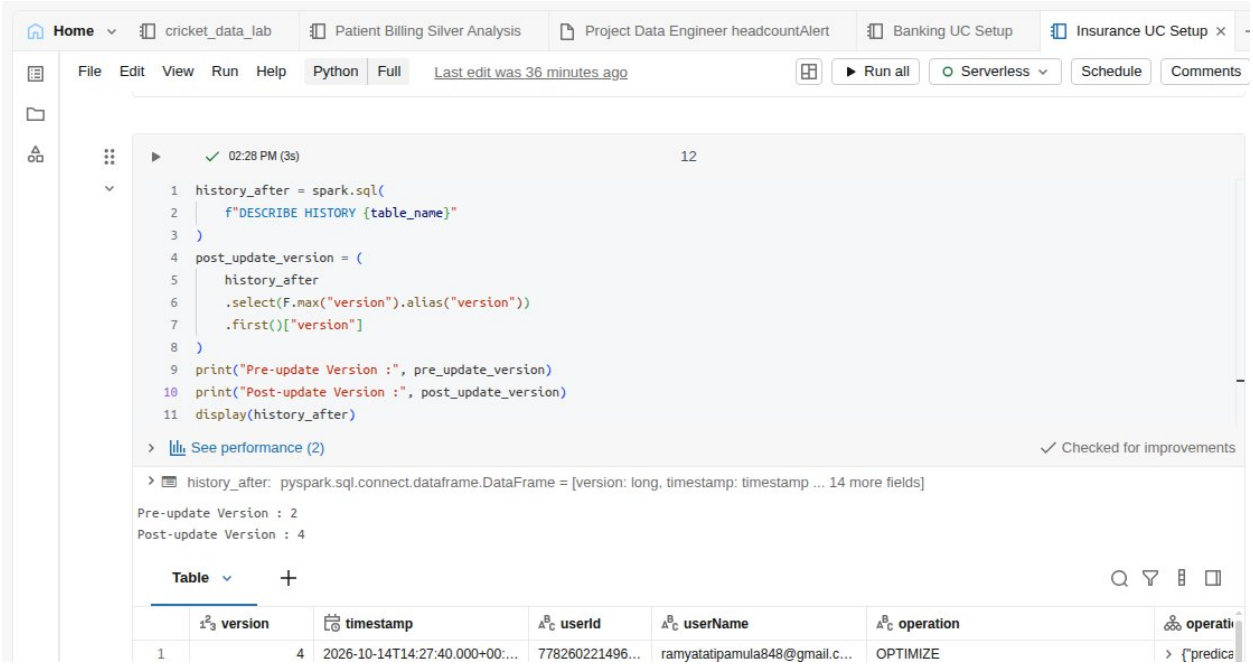

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking UC Setup
Insurance UC Setup
File Edit View Run Help Python Full
Last edit was 32 minutes ago
Run all
Serverless
Schedule
Comments
Table
claim_id | policy_id | claim_type | claim_amount | approved_amount | claim_status | claim_date
1 CL001 P101 Health 18000 15000 APPROVED 2026-09-01
2 CL002 P102 Vehicle 9000 0 REJECTED 2026-09-01
3 CL003 P103 Health 25000 20000 APPROVED 2026-09-02
4 CL005 P105 Vehicle 12000 10000 APPROVED 2026-09-03
5 CL006 P106 Property 35000 30000 APPROVED 2026-09-03
6 CL007 P107 Health 8000 0 PENDING 2026-09-04
7 CL008 P108 Vehicle 22000 18000 APPROVED 2026-09-04
8 CL009 P109 Property 16000 0 REJECTED 2026-09-05
9 CL010 P110 Health 28000 24000 APPROVED 2026-09-05
9 rows 3.552s runtime
Refreshed at 10/14/26, 2:25 PM (36m ago)
02:25 PM (3s)
8
%sql
ALTER TABLE uc_practice_insurance.claims_ops.claims_silver
SET TBLPROPERTIES (
delta.enableChangeDataFeed = true
);

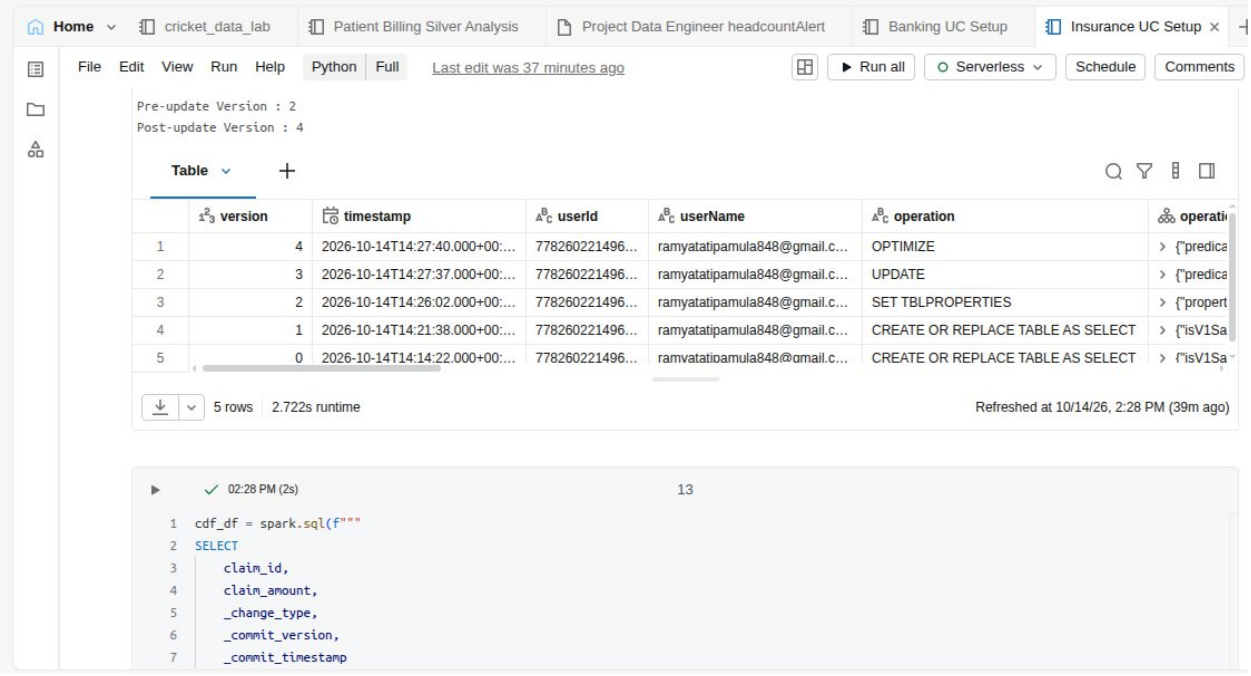

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking UC Setup
Insurance UC Setup
File Edit View Run Help Python Full
Last edit was 33 minutes ago
Run all
Serverless
Schedule
Comments
See performance (1)
Checked for improvements
OK
02:26 PM (3s)
9
%sql
DESCRIBE HISTORY
uc_practice_insurance.claims_ops.claims_silver;
See performance (1)
Checked for improvements
Table
version | timestamp | userId | userName | operation | operation
1 2 2026-10-14T14:26:02.000+00:... 778260221496... ramyatatipamula848@gmail.c... SET TBLPROPERTIES {"propertie
2 1 2026-10-14T14:21:38.000+00:... 778260221496... ramyatatipamula848@gmail.c... CREATE OR REPLACE TABLE AS SELECT {"isV1Save
3 0 2026-10-14T14:14:22.000+00:... 778260221496... ramyatatipamula848@gmail.c... CREATE OR REPLACE TABLE AS SELECT {"isV1Save
3 rows 3.277s runtime
Refreshed at 10/14/26, 2:26 PM (36m ago)
This result is stored as _sqldf and can be used in other Python and SQL cells.

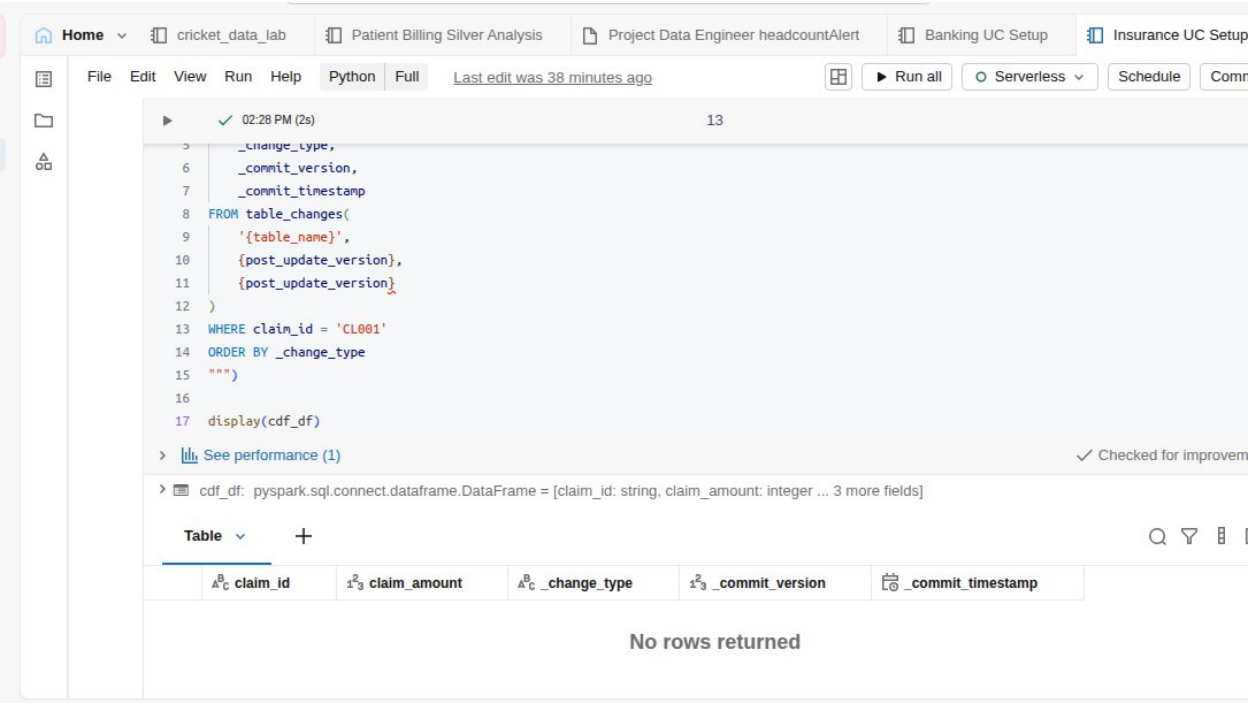

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking UC Setup
Insurance UC Setup
File Edit View Run Help Python Full
Last edit was 34 minutes ago
Run all
Serverless
Schedule
Comments
Share
02:27 PM (3s)
10
table_name = "uc_practice_insurance.claims_ops.claims_silver"
history_before = spark.sql(
f"DESCRIBE HISTORY {table_name}"
)
pre_update_version = (
history_before
.select(F.max("version").alias("version"))
.first()["version"]
)
print("Pre-update Version :", pre_update_version)
display(
spark.sql(f"""
SELECT
claim_id,
claim_amount
FROM {table_name}
WHERE claim_id = 'CL001'
""")
)
See performance (2)
Checked for improvements
history_before: pyspark.sql.connect.dataframe.DataFrame = [version: long, timestamp: timestamp ... 14 more fields]
Pre-update Version : 2

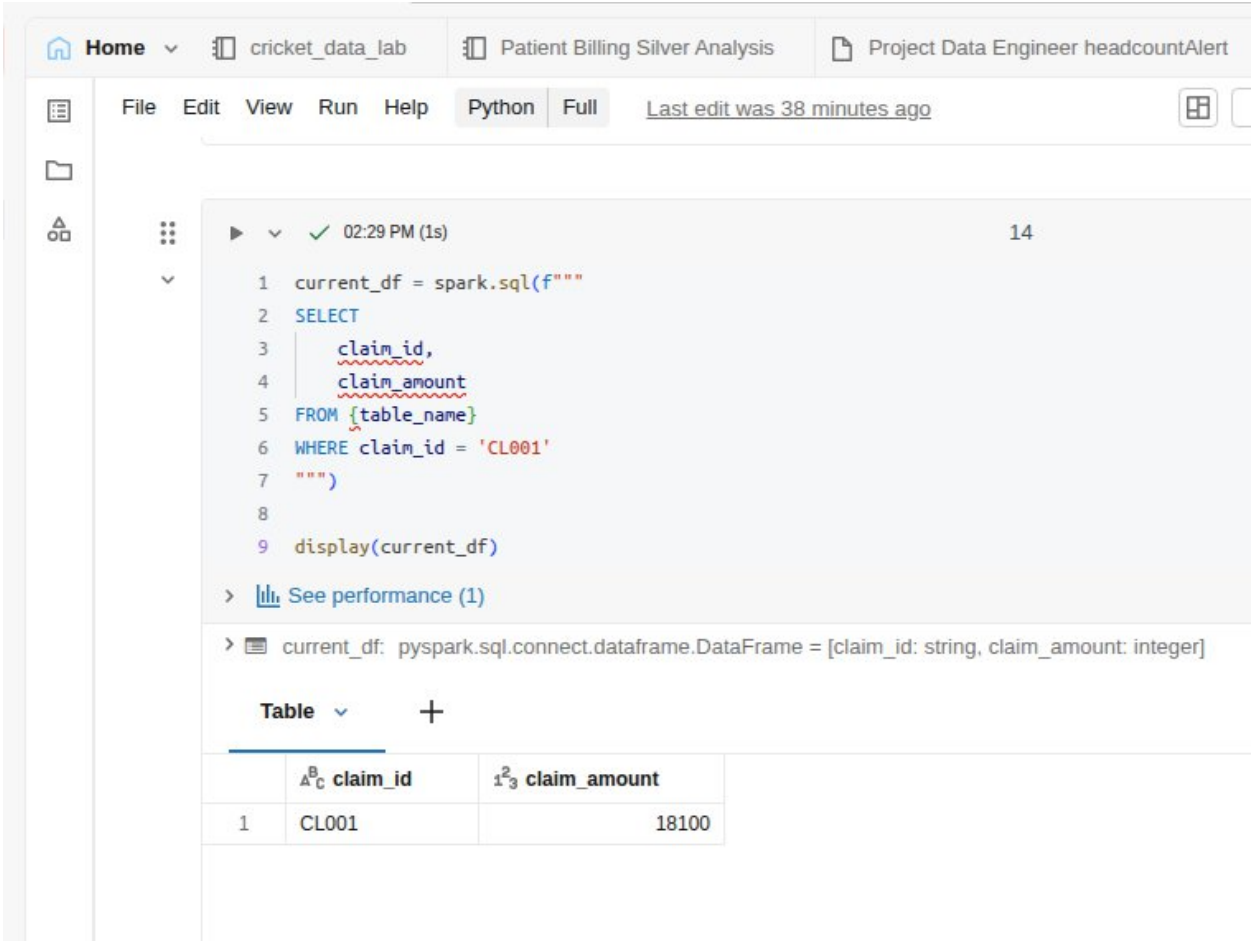

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking UC Setup
Insuranc
File Edit View Run Help Python Full
Last edit was 35 minutes ago
Run all
Serverless
history_before: pyspark.sql.connect.dataframe.DataFrame = [version: long, timestamp: timestamp ... 14 more fields]
Pre-update Version : 2
Table
claim_id
claim_amount
1
CL001
18000
1 row | 3.137s runtime
Refreshed at 10/14/26, 2:
02:27 PM (7s)
11
SQL
%sql
UPDATE uc_practice_insurance.claims_ops.claims_silver
SET claim_amount = claim_amount + 100
WHERE claim_id = 'CL001';
See performance (1)

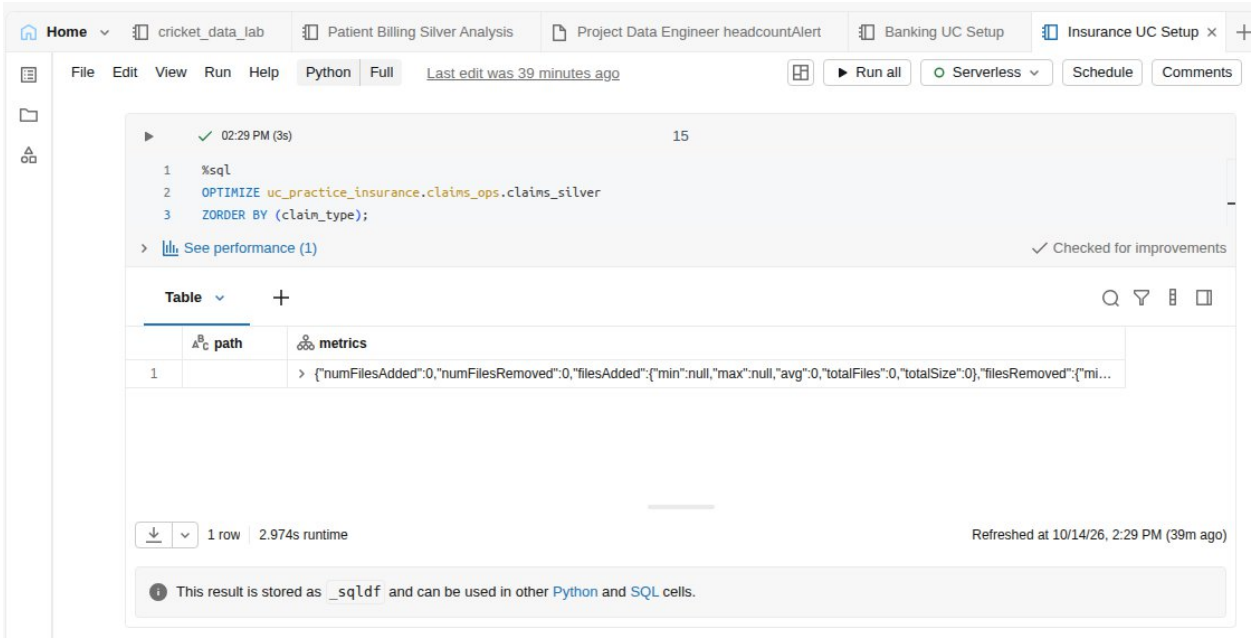

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking
File Edit View Run Help Python Full
Last edit was 35 minutes ago
Run all
02:27 PM (7s)
11
%sql
UPDATE uc_practice_insurance.claims_ops.claims_silver
SET claim_amount = claim_amount + 100
WHERE claim_id = 'CL001';
See performance (1)
Table
num_affected_rows
1
1 row
6.883s runtime
This result is stored as _sqldf and can be used in other Python and SQL cells.

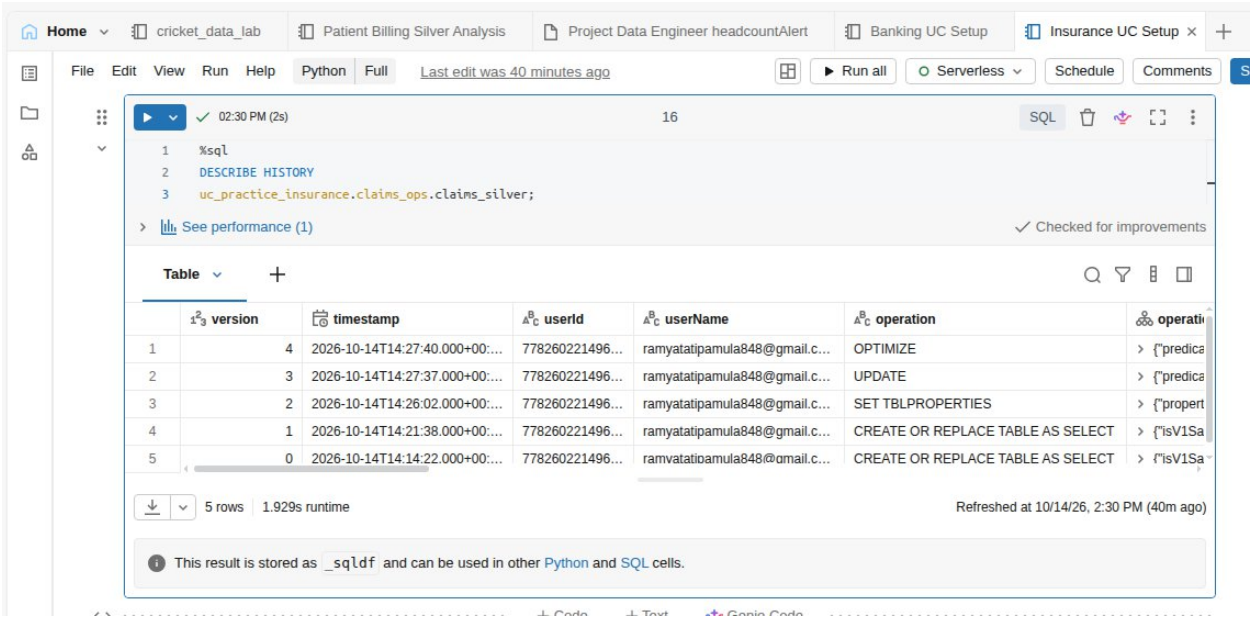

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking UC Setup
Insurance UC Setup
File Edit View Run Help Python Full
Last edit was 36 minutes ago
Run all
Serverless
Schedule
Comments
02:28 PM (3s)
12
history_after = spark.sql(
f"DESCRIBE HISTORY {table_name}"
)
post_update_version = (
history_after
.select(F.max("version").alias("version"))
.first()["version"]
)
print("Pre-update Version :", pre_update_version)
print("Post-update Version :", post_update_version)
display(history_after)
See performance (2)
Checked for improvements
history_after: pyspark.sql.connect.dataframe.DataFrame = [version: long, timestamp: timestamp ... 14 more fields]
Pre-update Version : 2
Post-update Version : 4
Table
version
timestamp
userId
userName
operation
1
4
2026-10-14T14:27:40.000+00:...
778260221496...
ramyatatipamula848@gmail.c...
OPTIMIZE

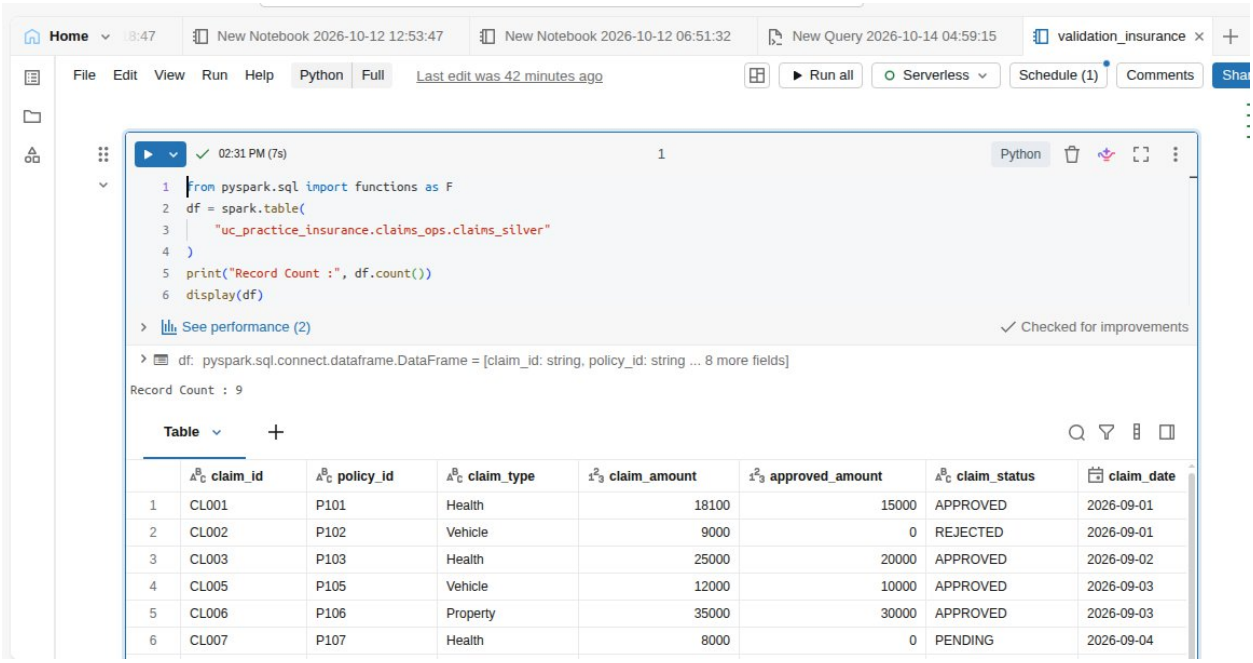

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking UC Setup
Insurance UC Setup
File Edit View Run Help Python Full Last edit was 37 minutes ago
Run all Serverless Schedule Comments
Pre-update Version : 2
Post-update Version : 4
Table
version timestamp userId userName operation
1 4 2026-10-14T14:27:40.000+00:... 778260221496... ramyatatipamula848@gmail.c... OPTIMIZE
2 3 2026-10-14T14:27:37.000+00:... 778260221496... ramyatatipamula848@gmail.c... UPDATE
3 2 2026-10-14T14:26:02.000+00:... 778260221496... ramyatatipamula848@gmail.c... SET TBLPROPERTIES
4 1 2026-10-14T14:21:38.000+00:... 778260221496... ramyatatipamula848@gmail.c... CREATE OR REPLACE TABLE AS SELECT
5 0 2026-10-14T14:14:22.000+00:... 778260221496... ramyatatipamula848@gmail.c... CREATE OR REPLACE TABLE AS SELECT
5 rows 2.722s runtime
Refreshed at 10/14/26, 2:28 PM (39m ago)
02:28 PM (2s)
13
cdf_df = spark.sql(f"""
SELECT
claim_id,
claim_amount,
_change_type,
_commit_version,
_commit_timestamp

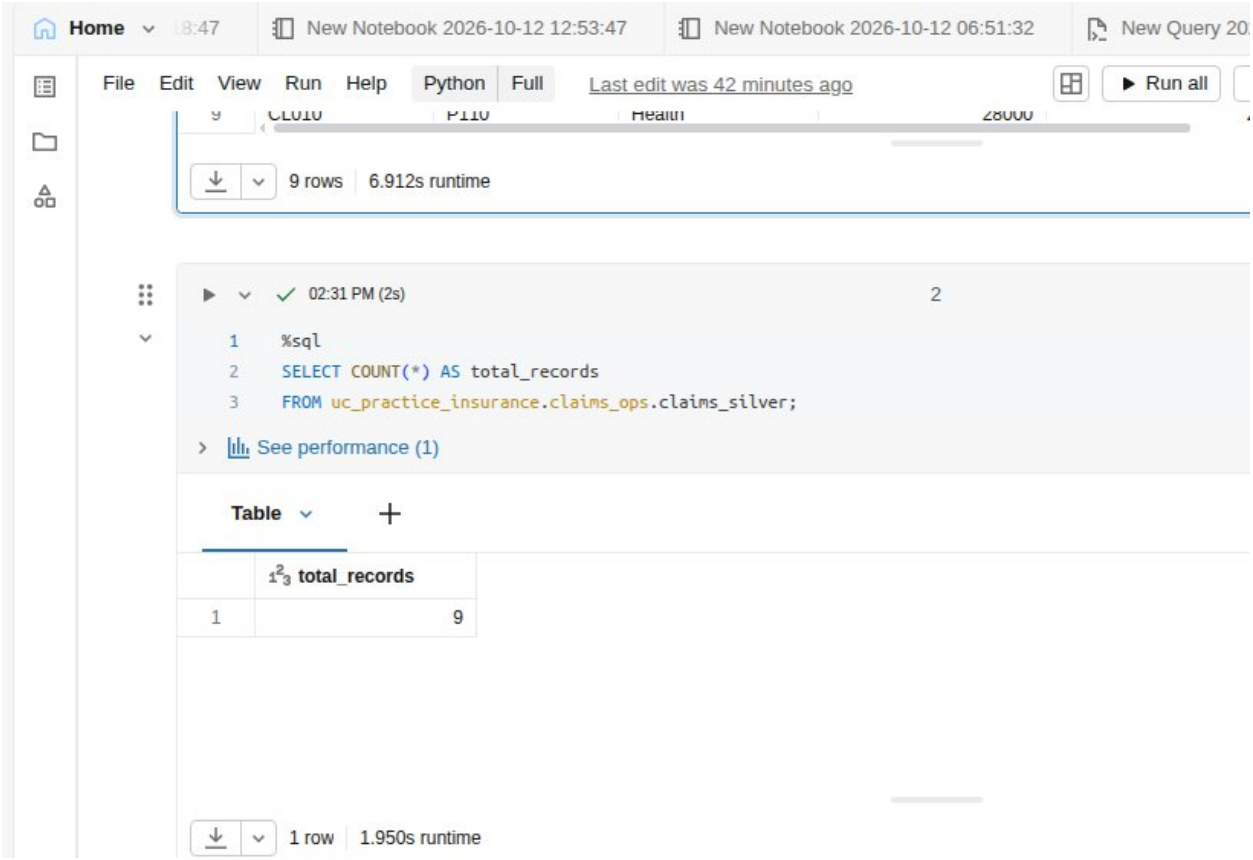

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking UC Setup
Insurance UC Setup
File Edit View Run Help Python Full Last edit was 38 minutes ago
Run all Serverless Schedule
02:28 PM (2s)
13
_commit_version,
_commit_timestamp
FROM table_changes(
'{table_name}',
{post_update_version},
{post_update_version}
)
WHERE claim_id = 'CL001'
ORDER BY _change_type
""")
display(cdf_df)
See performance (1)
Checked for improvem
cdf_df: pyspark.sql.connect.dataframe.DataFrame = [claim_id: string, claim_amount: integer ... 3 more fields]
Table
claim_id claim_amount _change_type _commit_version _commit_timestamp
No rows returned

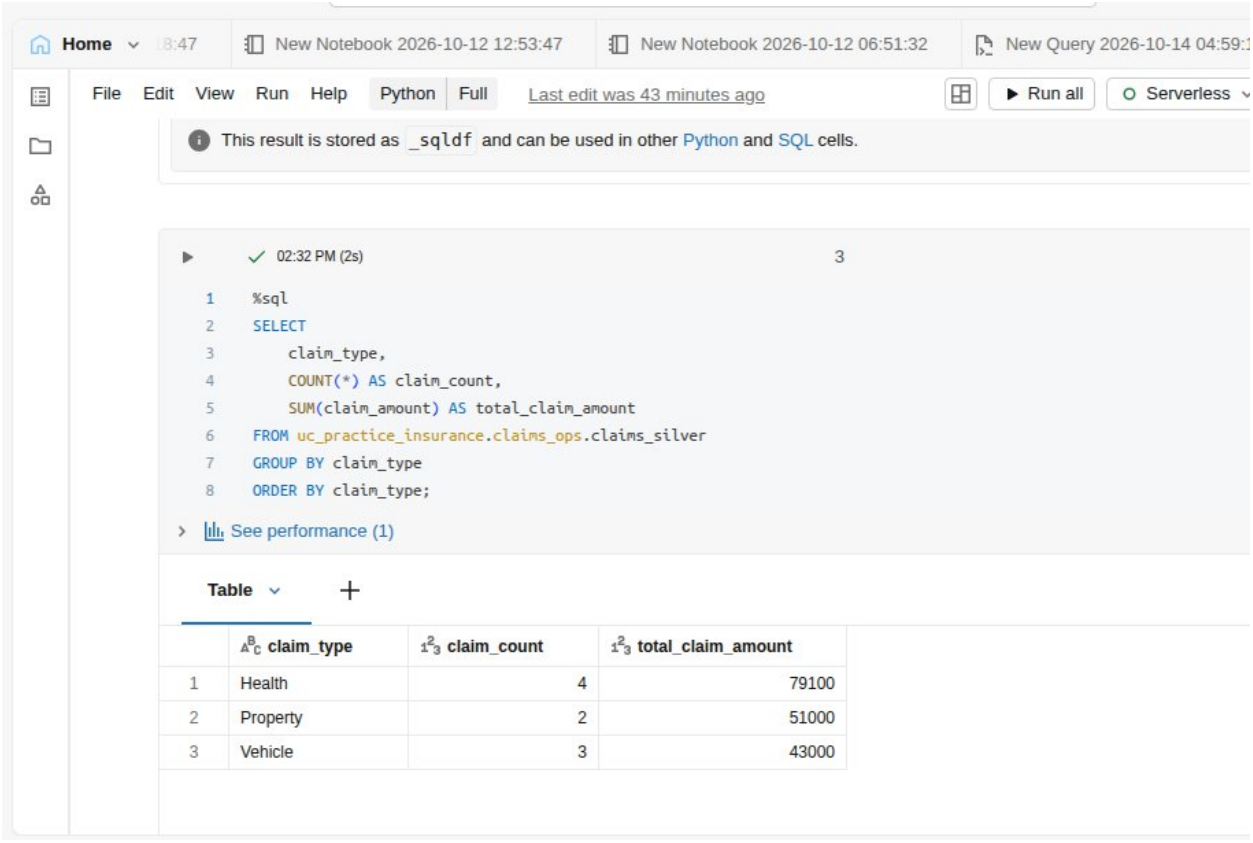

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
File Edit View Run Help Python Full
Last edit was 38 minutes ago
02:29 PM (1s)
14
current_df = spark.sql(f"""
SELECT
claim_id,
claim_amount
FROM {table_name}
WHERE claim_id = 'CL001'
""")
display(current_df)
See performance (1)
current_df: pyspark.sql.connect.dataframe.DataFrame = [claim_id: string, claim_amount: integer]
Table
claim_id
claim_amount
1
CL001
18100

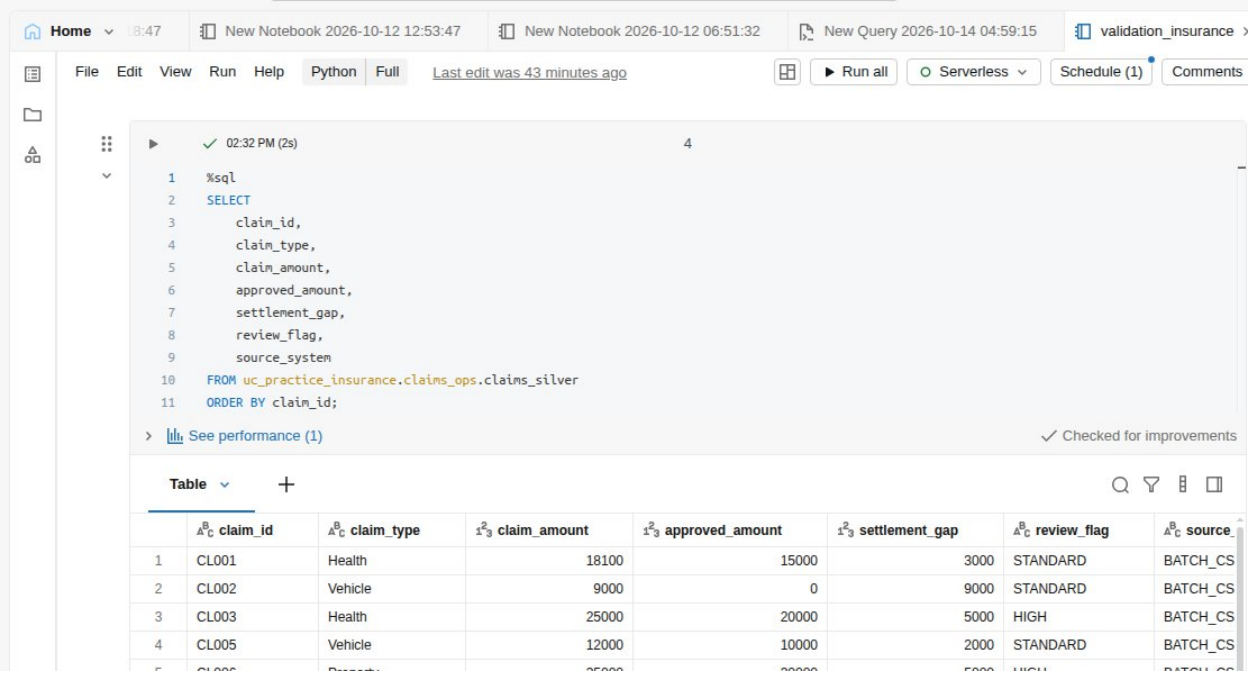

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking UC Setup
Insurance UC Setup
File Edit View Run Help Python Full
Last edit was 39 minutes ago
Run all
Serverless
Schedule
Comments
02:29 PM (3s)
15
%sql
OPTIMIZE uc_practice_insurance.claims_ops.claims_silver
ZORDER BY (claim_type);
See performance (1)
Checked for improvements
Table
path
metrics
1
{"numFilesAdded":0,"numFilesRemoved":0,"filesAdded":{"min":null,"max":null,"avg":0,"totalFiles":0,"totalSize":0},"filesRemoved":{"mi…
1 row
2.974s runtime
Refreshed at 10/14/26, 2:29 PM (39m ago)
This result is stored as _sqldf and can be used in other Python and SQL cells.

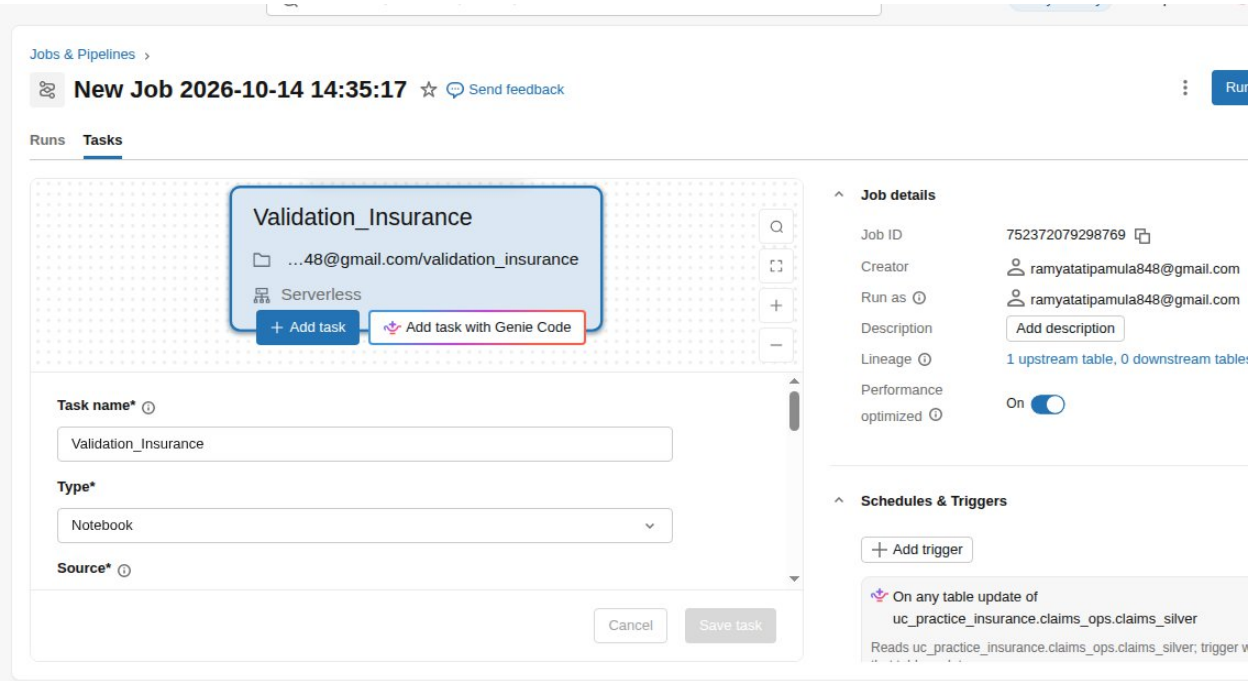

Home
cricket_data_lab
Patient Billing Silver Analysis
Project Data Engineer headcountAlert
Banking UC Setup
Insurance UC Setup
File Edit View Run Help Python Full Last edit was 40 minutes ago
Run all Serverless Schedule Comments
02:30 PM (2s) 16 SQL
%sql
DESCRIBE HISTORY
uc_practice_insurance.claims_ops.claims_silver;
See performance (1) Checked for improvements
Table
version timestamp userId userName operation
1 4 2026-10-14T14:27:40.000+00:... 778260221496... ramyatatipamula848@gmail.c... OPTIMIZE
2 3 2026-10-14T14:27:37.000+00:... 778260221496... ramyatatipamula848@gmail.c... UPDATE
3 2 2026-10-14T14:26:02.000+00:... 778260221496... ramyatatipamula848@gmail.c... SET TBLPROPERTIES
4 1 2026-10-14T14:21:38.000+00:... 778260221496... ramyatatipamula848@gmail.c... CREATE OR REPLACE TABLE AS SELECT
5 0 2026-10-14T14:14:22.000+00:... 778260221496... ramyatatipamula848@gmail.c... CREATE OR REPLACE TABLE AS SELECT
5 rows 1.929s runtime
Refreshed at 10/14/26, 2:30 PM (40m ago)
This result is stored as _sqldf and can be used in other Python and SQL cells.


Home
New Notebook 2026-10-12 12:53:47
New Notebook 2026-10-12 06:51:32
New Query 2026-10-14 04:59:15
validation_insurance
File Edit View Run Help Python Full Last edit was 42 minutes ago
Run all Serverless Schedule (1) Comments
02:31 PM (7s) 1 Python
from pyspark.sql import functions as F
df = spark.table(
"uc_practice_insurance.claims_ops.claims_silver"
)
print("Record Count :", df.count())
display(df)
See performance (2) Checked for improvements
df: pyspark.sql.connect.dataframe.DataFrame = [claim_id: string, policy_id: string ... 8 more fields]
Record Count : 9
Table
claim_id policy_id claim_type claim_amount approved_amount claim_status claim_date
1 CL001 P101 Health 18100 15000 APPROVED 2026-09-01
2 CL002 P102 Vehicle 9000 0 REJECTED 2026-09-01
3 CL003 P103 Health 25000 20000 APPROVED 2026-09-02
4 CL005 P105 Vehicle 12000 10000 APPROVED 2026-09-03
5 CL006 P106 Property 35000 30000 APPROVED 2026-09-03
6 CL007 P107 Health 8000 0 PENDING 2026-09-04


Home
New Notebook 2026-10-12 12:53:47
New Notebook 2026-10-12 06:51:32
File Edit View Run Help Python Full
Last edit was 42 minutes ago
Run all
9 rows | 6.912s runtime
02:31 PM (2s)
2
%sql
SELECT COUNT(*) AS total_records
FROM uc_practice_insurance.claims_ops.claims_silver;
See performance (1)
Table
total_records
1
9
1 row | 1.950s runtime


Home
New Notebook 2026-10-12 12:53:47
New Notebook 2026-10-12 06:51:32
New Query 2026-10-14 04:59:1
File Edit View Run Help Python Full
Last edit was 43 minutes ago
Run all
Serverless
This result is stored as _sqldf and can be used in other Python and SQL cells.
02:32 PM (2s)
3
%sql
SELECT
claim_type,
COUNT(*) AS claim_count,
SUM(claim_amount) AS total_claim_amount
FROM uc_practice_insurance.claims_ops.claims_silver
GROUP BY claim_type
ORDER BY claim_type;
See performance (1)
Table
claim_type
claim_count
total_claim_amount
1 Health 4 79100
2 Property 2 51000
3 Vehicle 3 43000


Home
New Notebook 2026-10-12 12:53:47
New Notebook 2026-10-12 06:51:32
New Query 2026-10-14 04:59:15
validation_insurance
File Edit View Run Help Python Full
Last edit was 43 minutes ago
Run all
Serverless
Schedule (1)
Comments
02:32 PM (2s)
4
%sql
SELECT
claim_id,
claim_type,
claim_amount,
approved_amount,
settlement_gap,
review_flag,
source_system
FROM uc_practice_insurance.claims_ops.claims_silver
ORDER BY claim_id;
See performance (1)
Checked for improvements
Table
claim_id
claim_type
claim_amount
approved_amount
settlement_gap
review_flag
source_
1 CL001 Health 18100 15000 3000 STANDARD BATCH_CS
2 CL002 Vehicle 9000 0 9000 STANDARD BATCH_CS
3 CL003 Health 25000 20000 5000 HIGH BATCH_CS
4 CL005 Vehicle 12000 10000 2000 STANDARD BATCH_CS


Jobs & Pipelines
New Job 2026-10-14 14:35:17
Send feedback
Run
Runs
Tasks
Validation_Insurance
...48@gmail.com/validation_insurance
Serverless
Add task
Add task with Genie Code
Task name*
Validation_Insurance
Type*
Notebook
Source*
Cancel
Save task
Job details
Job ID 752372079298769
Creator ramyatatipamula848@gmail.com
Run as ramyatatipamula848@gmail.com
Description Add description
Lineage 1 upstream table, 0 downstream tables
Performance optimized On
Schedules & Triggers
Add trigger
On any table update of
uc_practice_insurance.claims_ops.claims_silver
Reads uc_practice_insurance.claims_ops.claims_silver; trigger w